



## Objective:

Create a **React.js** ([BONUS-1] Use Next Typescript) application that renders items and has a login page.

If you're applying only for a Frontend position then you can ignore tasks marked as [BE]. For tasks/points marked as [FE] there will be an equivalent [BE] tag. Applicants applying for full stack roles need to do the [BE] task and [FE] task can be ignored

## Part 1: Rendering JSON Data

### Part 1.1

- Render the items
  - [FE] Render the given JSON file in a well designed and responsive UI. You are free to use any color scheme/design. Kindly do not use pre-existing components from libraries like React-MUI, Chakra, etc.
  - [BE] You will call your model viewset APIs to fetch and edit all items rather than using the JSON file. Look below in *Part 1.2* to understand the API requirements.
- The code should be able to dynamically handle all data so do not hard-code the values. Testing json file will be run separately so we should be able to drop in the replacement JSON and the UI should render properly.
- Provide the ability to edit the data from the UI.
- Use styling module, SCSS will be preferable.

### Part 1.2 [BE]

- Create a Django project and install Django Rest Framework.
- Create an abstract model called **BaseModel** with the following properties

```
- id: uuidv4
- created_at: automatically added DateTimeField at time of row creation
- updated_at: automatically added(at time of creation) and updated(at time of row updated) DateTimeField
```

- Create a model called **Items** which extends the abstract **BaseModel** with the following properties

- key: character field (50 chars maximum) cannot be null or blank
- value: string field

- Dump the shared json file to the database Items and create a DRF Model viewset to expose all CRUD APIs automatically
- [BONUS-2] Use PostgreSQL as the database.
- [BONUS-3] Use docker-compose.yaml to spin up your db and attach volumes to it so that the data stays persistent.

## Data Explanation

The JSON file contains several entries, each with various fields. The key fields of importance are "key" and "value." The "key" serves as the title for each entry, while the "value" represents the actual data.

## Part 2: API Calls & API Authentication

### Part 2.1

- Create a simple login page which will hit `{{BASE_URL}}/auth/token/` endpoint. The body of the request will look something like this. Kindly note for Frontend only positions, use dummy apis:

Request

```
{
  "username": "bm",
  "password": "kk"
}
```

Response

```
{
  "refresh":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...eyJ0b2t1b190eXBu0iw",
  "access":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...eyJ0b2t1b190eXB1Iknou"
}
```

- Write a custom React hook (`useApi`) to handle all API requests.
- The hook will add the `access` token `jwt` to all the api calls automatically in the header as `Authorization: Bearer <token>`.
- If the reply of the API call fails due to token expiry then it should be able to retry the request after renewing the token from auth API. The body of the refresh endpoint looks like this. Not that the refresh token is the same refresh token received in the response above

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...eyJ0b2t1bGl90eXBU0iw"
}
```

- Ensure the hook can be easily used throughout the project.

## Part 2.2 [BE]

- Integrate [SimpleJWT](#) in the Django Project.
- The authentication endpoint should be `{{BASE_URL}}/auth/token/` and the refresh token endpoint should be `/{{BASE_URL}}/auth/token/refresh/`

## Deliverables

- A **README.md** file explaining:
  1. How to run the project.
  2. How the custom hook works.
  3. Any assumptions made or additional features implemented.
- **[BE]** Properly structured Django & DRF code.

## [FE]

1. A React.js application that meets the above requirements.
2. The custom hook (`useApi`) implemented and integrated into the application.

### [FE] Bonus Points

1. Implement unit tests for the custom hook.
2. Use TypeScript for type safety.
3. Implement state management using a library like Redux or Zustand.

## Submission

- Push your code to a public GitHub repository.
- Maintain commit history in git. (do not push whole project in one go)
- Share the repository link along with any necessary instructions to run the project.
- [BE] [BONUS-4] use .env file to initialise passwords and secret keys. Since this is a dummy project, the .env file should be present in the repository as well

### Evaluation Criteria

- Code quality and readability.
- Proper usage of React and modern JavaScript/TypeScript features.
- Functionality and usability of the custom hook.
- Styling and responsiveness of the application.
- Documentation and adherence to the requirements.